

PHP

Rev 6.5 del 07/03/2026

Introduzione ai web server	2
Sintassi base di PHP	3
Funzioni	5
Array Enumerativi	7
Array Associativi	7
Vettori di json	9
Matrici	10
La gestione delle date	11
Variabili superglobali	12
Incorporazione di altri file: include e require	12
Connessione tra PHP e MySQL	13
L'oggetto mysqli	13
Serializzazione del recordset restituito da mysqli	15
Applicazioni SSR e Applicazioni CSR	16
Applicazioni SSR : Concetto di Web Form e Submit	16
La trasmissione dei parametri nelle web form	17
Esecuzione del submit tramite javascript	17
Lettura lato server dei Parametri Get e Post	18
Redirect	18
Applicazioni CSR : Creazione di un servizio	19
Principali Status Code ritornati dalle http Response	20
Gestione Caratteri speciali e apici singoli / doppi	20
Il problema dell'SQL injections	21
L'oggetto SESSION	22
La funzione md5()	25
Invio di una mail	26
Principali funzioni per la gestione di un file	29
Upload di un file binario	30
Lettura dei parametri passati con i metodi PUT PATCH e DELETE	32
Creazione e Serializzazione di uno stream XML	33
Debug di una applicazione PHP con vscode	35

Introduzione ai web server

Un **web server** è una applicazione che, a fronte di una precisa richiesta basata sul protocollo **http**, restituisce al client la risorsa richiesta che può essere

- un **file** (html, js, css, jpg)
- un **insieme di dati** in formato json

Il web server, prima di inviare la risposta al client, può eseguire delle **elaborazioni** sulle pagine e/o sui dati

Quando il client riceve dal server una pagina **html**, provvede a 'leggerla' e a richiedere al server, una alla volta, tutte le ulteriori risorse utilizzate da quella pagina: .css .js, immagini, video, file di caratteri .ttf

Pagine Dinamiche

Le attuali pagine html sono sempre pagine **dinamiche**, nel senso che devono aggiornare il contenuto più o meno frequentemente. Si pensi ad esempio a:

- una pagina di previsioni meteo, in cui la pagina è sempre la stessa ma i dati cambiano
- un catalogo prodotti, in cui i dati cambiano meno frequentemente ma devono essere aggiornati
- una pagina contenente le quotazioni di borsa, in cui i dati cambiano in modo continuo.

In tutti questi casi il web server, prima di inviare la pagina al client, deve aggiornare i dati.

A tal fine storicamente si sono succedute due modalità differenti:

- **Modalità orientata alle pagine** (o **SSR = Server Side Rendering**) in cui il server, in corrispondenza di ogni richiesta, ricostruisce dinamicamente l'intera pagina html prima di inviarla al client, aggiornando eventualmente i dati letti da database
- **Modalità orientata ai servizi** (o **CSR = Client Side Rendering**) in cui il server invia inizialmente una pagina html statica priva di dati e poi tutta la costruzione della pagina è demandata al client che richiederà via via i dati necessari tramite successive chiamate **Ajax**. Molto più leggero perché la pagina html non viene più ricaricata.

Le principali tecnologie web lato server

Tecnologia	Web server	Organizzazione	Database
JAVA	Tomcat	Apache Foundation	Oracle DB
PHP	Apache	Apache Foundation	MySQL
.NET	IIS	Microsoft	SQL Server
Nodejs	Express	Node.js Foundation	MongoDB
Python	Django	Python Software Foundation	PostgreSQL

Apache Foundation è uno storico gestore di progetti open source utilizza GNU License (GNU's Not Unix)

Il marchio MySQL è stato acquistato da Oracle, per cui nelle distribuzioni PHP è stata introdotta una fork denominata MariaDB

Cosa fa PHP

php è una sigla ricorsiva che significa **Php Hypertext PreProcessor** cioè **preprocessore testuale php**. La tecnologia PHP consiste nell'inserire degli **script php** all'interno di una pagina html.

Il server, prima di inviare la pagina al client, la legge e traduce questi script in normali tag HTML così che il browser riceverà sempre solo pagine HTML.

- Le pagine con estensione **.html** non possono contenere **script php** vengono inviate al client così come sono.
- Le pagine con estensione **.php** sono normali pagine html che però possono contenere degli script php che vengono eseguiti dal server prima di inviare la pagina al client.

Utilizzo di xampp

XAMPP è un porting su windows di un framework nato come **LAMPP** in ambiente Linux. I server web sono spesso basati su linux per ragioni di sicurezza. Però spesso aziende e programmatori utilizzano windows per cui la piattaforma lamp è stata adattata anche per funzionare in ambiente windows.

X = Windows
A = Apache (web server)
M = MySQL (database di tipo client server)
P = PERL (linguaggio server alternativo a PHP)
P = PHP

Avvio di Apache

L'installer provvede ad installare l'intero pacchetto all'interno della cartella **c:\xampp**.

La sottocartella **htdocs** rappresenta la cartella home per le applicazioni utente

Il pannello **xampp\xampp-control.exe** consente di avviare il web server Apache sulla porta indicata all'interno del file Config / apache, accessibile tramite apposito **pulsante in alto a destra**

Config / Service and Port Settings. Per modificare la porta di ascolto aprire il file, cercare 80 e sostituirlo con la porta desiderata (2 sostituzioni).

Clickando su **Admin** si apre la pagina informativa C:\xampp\htdocs\dashboard\index.html

Sintassi base di PHP

Case Sensitive. Non accetta caratteri speciali al di fuori dell'underscore

Ogni istruzione **deve** essere terminata da ; esattamente come in C

Virgolette doppie e semplici possono essere utilizzate quasi indifferentemente (Le virgolette doppie accettano variabili al loro interno, le virgolette singole no).

```
/* Commento multi linea */  
// Commento su singola riga  
# Commento nativo php, poco utilizzato
```

Gli script php possono essere inseriti in qualsiasi punto di una pagina html, che **però** deve necessariamente avere estensione **.php**.

echo () con o senza parentesi tonde, consente di scrivere sulla pagina html prima che questa venga inviata al client. Accetta al suo interno una stringa contenente qualsiasi tag HTML.

La funzione **echo ()** non va a capo automaticamente. Poiché va a scrivere all'interno del file HTML, per andare a capo occorre utilizzare il tag "**
" e non "\n**"

```
echo phpversion(); // visualizza la versione corrente di php 11.2 ad aprile 2025
```

die ("msg") Analogo a **echo** però **termina l'elaborazione dello script**. Comodo per segnalare ad esempio situazioni di errore. L'utente premerà indietro e ritornerà alla pagina precedente.

Nel caso di **die ()** le parentesi tonde sono obbligatorie.

print () è identica a **echo** ma in più restituisce true / false a seconda che l'operazione sia andata a buon fine. In pratica restituisce sempre true, ma il fatto di ritornare un valore consente di utilizzarla all'interno dell'operatore condizionale ternario: `$b ? print "true" : print "false";` Però è più lenta.

print_r () serializza qualunque variabile visualizzandola nella pagina in un formato human-readable. Ad esempio serializza automaticamente vettori sia enumerativi che associativi. Presenta anche un secondo parametro per definire il formato di visualizzazione dei dati. **Usata di solito dentro echo()**

Dichiarazione ed utilizzo delle Variabili

In PHP variabili **devono** iniziare con il carattere \$. Ad esempio: \$n = 3.4; \$NumeroEsa = 0x1B

- Non è richiesta la dichiarazione. Le variabili vengono allocate **automaticamente** al primo utilizzo.
- Le variabili non sono tipizzate, esattamente come in javascript. Converti in stringa e poi confronta `if($a==5)` equivale a `if($a=="5")`. Per fare il controllo sul tipo si usano i tre uguali (`$a===5`)
- **Le variabili dichiarate all'interno di uno script sono visibili in tutti gli altri script della pagina, ma non sono visibili all'interno di eventuali funzioni utente.** (se non sono dichiarate **global**)

Per vedere se una variabile esiste : `if(isset($var))`

Per vedere se una variabile è stata valorizzata : `if($var)` oppure `if(!empty($var))`

La prima va in errore se `$var` non è definita, la seconda no. In php non esiste `undefined`

Costanti

```
define ("NOME", "Mario Rossi"); // oppure
const NOME = "Mario Rossi";
```

La seconda sintassi è più moderna ma non può essere usata dentro una if oppure dentro un ciclo

Stringhe

- All'interno di una stringa è **sempre possibile andare a capo**, sia all'interno degli apici singoli sia all'interno degli apici doppi
- All'interno delle stringhe è possibile utilizzare variabili semplici
`$s = "Salve $nome !"; // Salve Pippo !.`
 Attenzione però che funziona solo all'interno delle **virgolette doppie**.
 Gli apici singoli non interpretano le variabili.
- Nella versioni più recenti è stata introdotto l'utilizzo della parentesi graffe che consentono di valorizzare anche campi compositi e di risolvere espressioni php valide (es operazioni matematiche o accesso ad un valore di un array).

```
$msg = "Benvenuto {$person['nome']}" // All'interno bisogna utilizzare apici singoli !
$msg = "Benvenuto {$person[0]['nome']}";
$msg = "Età nel prossimo anno: {$person['eta'] + 1}";
```

La sintassi precedente (senza graffe) ammetteva soltanto singoli livelli di indicizzazione ed occorreva omettere gli apici intorno alla chiave

- Per il concatenamento si usa l'operatore `.` `$s = "mario" . "rossi";` E' ammesso anche `.=`

Booleani

Il **false** equivale a **0**. Tutto ciò che non è **0** è considerato **true**. Esattamente come in js

L'istruzione `if($s)` restituisce true se `$s` non contiene stringa vuota

```
$ris = $nome ?? "pippo"
```

Se `$nome` non esiste, `$ris` assume il valore `"pippo"`

Operatori

A parte il concatenamento, sono gli stessi identici del C

Aritmetici	+	-	*	/	% (resto della divisione fra interi)	++	--
Assegnazione	=	+=	-=	*=	/=		
Confronto	==	!=	>	>=	<	<=	
Logici	&&		!				

Caratteri di escape

Utilizzabili nelle stringhe. Sono gli stessi identici del C: `\n` `\t` `\\` `\"` `\$`
 Questi caratteri non sono interpretati dall'HTML, per cui non possono essere utilizzati quando si invia una risposta ad un browser. Possono essere utilizzati quando si salva qualcosa su file di testo.

Creazione di variabili a partire dal contenuto di una stringa

```
$var="txtNome";
$$var="pippo";    // crea una nuova variabile di nome txtNome
echo($txtNome);   // pippo
```

`$$var` crea una nuova variabile avente come nome `$txtNome` (cioè il contenuto della variabile `$var`)

La parola chiave global

Le funzioni utente **non 'vedono'** le variabili 'globali' dichiarate all'interno degli script (nemmeno quelle dichiarate all'interno dello stesso script). Una funzione può comunque accedere alle variabili dello script anteponendo al nome della variabile il termine **global**, che può però essere usato **soltanto** in fase dichiarativa. Tecnica alternativa rispetto al passaggio dei parametri, ma meno strutturata.

```
global $contatoreSomme = 0;    // qui global è facoltativo
function somma($a, $b) {
    $somma = $a + $b;
    global $contatoreSomme;
    $contatoreSomme++;
    return $somma;
}
```

Istruzioni Base

Stesse identiche di C# `if` `-` `for` `-` `do...while` `-` `while` `-` `switch`

Switch

```
switch ($colore) {
    case 'blu'      : echo ("Il colore selezionato è blu");      break;
    case 'giallo'   : echo ("Il colore selezionato è giallo");   break;
    default        : echo ("Nessun colore selezionato");        break;
}
```

Come in C, una volta individuato un case valido, l'elaborazione procede a meno che non ci sia un break.

Funzioni

L'utilizzo delle funzioni è simile a Java Script. Dallo script principale è possibile richiamare qualunque funzione (scritta prima o dopo non importa). Dall'interno di una funzione è possibile richiamare un'altra funzione ma solo se questa è scritta prima della funzione da cui viene richiamata.

Passaggio dei parametri

I parametri vengono passati di default per valore.

Per passare un parametro **per indirizzo** occorre anteporre l'operatore **&** **nella definizione della funzione** (al contrario dell'Ansi C). `elabora($val1, &$val2);` // il secondo parametro è passato per indirizzo
 Questo vale anche nella scansione di un array se occorre apportare modifiche all'elemento corrente :
`foreach ($people as &$person) {`

La prima pagina PHP

```
<?php
    $nome = "pippo";
```

```

echo("<p> Il mio nome è <b>$nome</b> <p>");
visualizza($nome);
function visualizza($nome) {
    echo("<p Ciao! Sono $nome e sono uno studente di Informatica </p>");
} ?>

```

Sono anche possibili istruzioni di questo tipo:

```

<?php if($flag): ?>
    <div> contenuto 1 </div>
<?php else: ?>
    <div> contenuto 2 </div>
<?php endif; ?>

<?php for($i=0; $i<100; $i++): ?>
    <div> contenuto <?php echo($i) ?> </div>
<?php endfor; ?>

```

Funzioni di libreria

isset(\$var) restituisce **true** se la variabile esiste (cioè se è già stata dichiarata o utilizzata)
 gettype(\$var) restituisce il tipo della variabile (boolean, integer, string, array, object)

Le seguenti funzioni restituiscono invece true/false:

is_string(\$var), is_numeric(\$var), is_int(\$var), is_float(\$var),
 is_double(\$var), is_bool(\$var), is_array(\$var), is_object(\$var),

matematiche

\$val = **rand**(A, B) genera un numero intero random compreso tra A e B entrambi gli estremi inclusi
 \$ris = **pow**(2, 3) Esegue l'elevamento a potenza 2³
 \$ris = **round**(num, nCifre) Arrotonda a n cifre dopo la virgola
is_numeric(\$str) Restituisce true se la stringa al suo interno contiene soltanto caratteri numerici

stringhe

\$n = **intval**(\$str) Converte la str in numero oppure semplicemente \$n = (**int**)\$str
 \$str = **strtoupper**(\$str) Converte in maiuscolo
 \$str = **strtolower**(\$str) Converte in minuscolo
 \$str = **ucwords**(\$str) - Uppercase the first character of each word in a string
 \$s = **str_replace**(\$s1, \$s2, \$string) Sostituisce s1 con s2 all'interno di string
 \$s = **substr**(\$string, \$start [, int \$n]) Estrae \$n caratteri a partire dalla posiz \$start
 \$pos = **strpos**(\$string, \$substring, [\$start])
 Restituisce la prima occorrenza di substring all'interno di string. Se non la trova restituisce false
 \$pos = **strrpos**(\$string, \$substring, [\$start])
 Restituisce l'ultima occorrenza di substring all'interno di string. Se non la trova restituisce false
stripos() e **strripos()** Uguali alle precedenti ma eseguono una ricerca case un sensitive
 \$str = **htmlentities**(\$str) Converte in htmlentities (> < ") tutti i caratteri che hanno un corrispondente carattere di tipo htmlentities. Usata per passare parametri testuali da una pagina all'altra in modo da convertire i caratteri speciali(ed gigi d'alessio)
 \$str = **htmlspecialchars**(\$str) Evoluzione della precedente che non converte tutti i caratteri che dispongono di un htmlentities, ma *solo* i caratteri speciali. **Non** converte ad esempio le lettere accentate.
 \$str = **nl2br**(\$str) Converte il \n in

Altre funzioni

<code>exit(0)</code>	funzione che termina l'elaborazione della pagina corrente 0 rappresenta il valore default, per cui può essere omesso.
<code>sleep(\$nSec)</code>	Mette lo script in attesa per n secondi
<code>usleep(\$nSec)</code>	Mette lo script in attesa per n micro secondi

Vettori Enumerativi

```
$vect = []; // oppure
$vect = array(); // senza il new !

$vect = ['a', 'b', 'c', 'd'];
$vect = array('a', 'b', 'c', 'd');

echo ($vect [3]); // d
```

Funzioni sui vettori enumerativi

count(\$vect); // Numero di elementi contenuti nel vettore
array_push(\$vect, "e", "f", "g"); // Consente di aggiungere nuove voci al vettore.
Le voci possono essere anche object, ma il vettore **deve** essere enumerativo e NON associativo
\$vect [] = "e"; // Analogo al precedente. Più moderno e preferibile.
Concatena una singola voce in coda al vettore.
\$massimo = max(\$vect); // Trova il max/min in un vettore numerico
array_sum(\$vect); // Somma gli elementi di un vettore numerico
is_array(\$vect) // Restituisce true/false se \$vect è un vettore enumerativo o associativo.

Conversione dei vettori enumerativi in stringa **implode()** e **explode()**

Sono l'equivalente di **split()** e **join()** in javascript.

explode() Converte una stringa in un vettore enumerativo come lo split di javascript
Non esiste in php un alias split(). Esiste in php una funzione **str_split()** ma fa una cosa leggermente diversa.
`$vect = explode("-", $str);`

implode() converte un vettore enumerativo in stringa come il join di javascript.
Non funziona con i vettori associativi. E' stato aggiunto anche un alias **join()**.
`$str = implode(",", $vect);`
`$str = join(",", $vect);`

In realtà un vettore enumerativo è un JSON valido, quindi può essere serializzato mediante il metodo **json_encode() come indicato nella pagina successiva.**

Scansione di un vettore enumerativo

```
$somma = 0;
$array = array(1, 2, 3, 4, 5);
for($i=0; $i<count($array); $i++) // oppure
foreach ($array as $item)
    $somma += $item;
```

Notare che `$item` viene sempre passato per valore. Per passarlo come riferimento occorre scrivere `foreach ($array as &$item)`

Vettori Associativi

Per creare un array associativo non esiste la classica notazione java script

```
let obj = { "nome": 'pippo', "eta": 16, "residenza": 'fossano' }
```

ma occorre utilizzare la seguenti sintassi:

```
$people = ["nome"=>'pippo', "eta"=>16, "residenza"=>'fossano']; // oppure  
$people = array("nome"=>'pippo', "eta"=>16, "residenza"=>'fossano');
```

cioè invece della graffa si usa la quadra, e al posto dei *due punti* si utilizza la freccina (double arrow)

Il double arrow operator consente di creare un vettore associativo anziché enumerativo, in cui ogni item sarà composta da una combinazione chiave => valore.

```
$people = ["Peter"=>35, "Ben"=>37, "Joe"=>43]; // oppure  
$people = array("Peter"=>35, "Ben"=>37, "Joe"=>43);
```

Notare che, a differenza di js, **gli apici intorno ai nomi delle chiavi sono obbligatori**.

In caso contrario non vengono generati errori in fase di scrittura, ma si genera un Run Time Error nel momento in cui l'istruzione viene eseguita.

In alternativa, come in js, è possibile dichiarare un vettore vuoto e poi popolarlo mediante chiavi successive:

```
$people = []; // oppure $people = array()  
$people['Peter'] = 35; $people['Ben'] = 37; $people['Joe'] = 43;
```

I campi di un vettore associativo possono essere acceduti in modo diretto ma soltanto usando la sintassi con le parentesi quadre:

```
$peterAge = $age["Peter"];
```

Non è ammessa la sintassi con il puntino `$peterAge = $age.Peter;`

Scansione di un vettore associativo

L'operatore => è utilizzabile anche all'interno di un ciclo foreach per scandire gli elementi di un vettore associativo. In tal caso consente di scorrere le varie chiavi ciascuna con il proprio valore:

```
foreach ($people as $key => $value) {  
    echo "L'utente " . $key . " ha " . $value . " anni\n";  
}
```

Quando si scandisce un vettore associativo è obbligatorio impostare **sia la chiave** **sia il valore**.

In caso contrario viene generato un errore di sintassi: `foreach($people as $key) // errore !`

Serializzazione di un vettore associativo

La funzione `json_encode($obj, JSON_PRETTY_PRINT)` consente di serializzare un JSON in stringa. Il secondo parametro fa sì che venga eseguita una formattazione più leggibile analogamente al valore di formattazione di `JSON.stringify(obj, null, 3)`

La funzione `json_decode($str, true)` parsifica una stringa JSON e restituisce un vettore associativo. Restituisce **null** in caso di stringa json NON valida.

Il secondo parametro (default `false`)

- se **true** fa sì che `json_decode` restituisca un **vettore associativo** in cui le chiavi saranno accessibili mediante indicizzazione con parentesi quadre.
- se **false** fa sì che `json_decode` restituisca un **PHP OBJECT** in cui le chiavi saranno accessibili mediante l'operatore `->`, come si accede normalmente alle proprietà di un Object

Nota:

Dato un vettore di stringhe,

`json_encode` restituisce `'["stringa1", "stringa2"]'`

`implode` restituisce invece `'stringa1, stringa2'`

Esempi:**1. Serializzazione di un vettore enumerativo**

```
$vect = array("Peter", "Ben", "Joe");  
$s = json_encode($vect);  
echo ($s); // ' ["Peter", "Ben", "Joe"] '
```

2. Serializzazione di un vettore associativo

```
$vect = array("Peter"=>35, "Ben"=>37, "Joe"=>43);  
$s = json_encode($vect);  
echo ($s); // '{"Peter":35, "Ben":37, "Joe":43}'
```

Vettori di JSON

```
$incarico1 = ["incarico"=>"docente",  
             "materia"=>"C++",  
             "inizio"=>2009,  
             "fine"=>2009,  
             "istituto"=>"StudiareInformatica s.n.c."];  
  
$incarico2 = array("incarico"=>"programmatore",  
                  "settore"=>"Java",  
                  "inizio"=>2008,  
                  "fine"=>2013,  
                  "azienda"=>"ABC s.r.l.");  
  
$vect = [$incarico1, $incarico2] // oppure  
$vect = array($incarico1, $incarico2)  
  
// trova e visualizza tutti i docenti  
foreach ($vect as $item) {  
    if (isset($item["incarico"]) && $item["incarico"] == "docente") {  
        echo json_encode($item);  
    }  
}
```

json annidati

// creare in php la seguente struttura javascript :

```
{ "posts" :  
  [ { "post": { "file" : "file1", "colore" : "colore1" } },  
    { "post": { "file" : "file2", "colore" : "colore2" } } ],  
}  
  
$data = [];  
$data["posts"] = [];
```

```
for ($i = 1; $i <= N; $i++) {
    $item = ["file" => "file{$i}", "colore" => "colore{$i}" ];
    $post = ["post" => $item] ;
    array_push($data["posts"], $post);    // oppure
    $data["posts"][] = $post;
}
```

Funzioni sui vettori associativi

array_merge concatena i due vettori ricevuti come parametro e restituisce il vettore risultante.

```
$age = array_merge($age, array("Jack"=>29, "foo"=>13));
```

In alternativa a `array_merge()` è anche possibile utilizzare semplicemente l'operatore `+`

```
$age = $age + array("Jack"=>29, "foo"=>13);
```

Conversione tra vettore associativo <-> urlEncoded

parse_str() converte una stringa urlEncoded in un vettore associativo. `parse_str($string, $vect);`

http_build_query() converte un vettore associativo in str urlEncoded. `$s=http_build_query($vect)`

Matrici

Sono vettori di vettori, esattamente come in javascript:

```
$cars = array (
    array("Volvo",22,18),
    array("BMW",15,13),
    array("Saab",5,2),
);

echo $cars[0][0]

for ($i=0; $i<count($cars); $i++) {
    echo "<ul>";
    for ($j=0; $j<count($cars[$i]); $j++)
        echo "<li>" . $cars[$i][$j] . "</li>";
    echo "</ul>"
}
```

Try and Catch

E' lo stesso di javascript:

```
try {
}
catch (Exception $ex) {
    echo($ex.getMessage());
}
```

Creazione di un file di log

```
$log = "risultato:" . $ris . PHP_EOL;    // oppure . "\n"
file_put_contents('./logFile.txt', $log, FILE_APPEND);
```

La gestione delle date

time() restituisce il timestamp unix corrente espresso in **sec** (numero di secondi trascorsi a partire dalla data zero dei sistemi unix, 1/1/1970)

microtime(true) Uguale alla precedente ma restituisce un float contenente anche i microsecondi. Impostando il parametro **true** il risultato viene restituito come float con i microsecondi aggiunti dopo la virgola. Impostando il parametro **false** (default) il risultato viene restituito come stringa nel formato "usec sec" con un semplice spazio come separatore

date(format, time) Converte un timestamp in una data di tipo stringa nel formato indicato **time**, facoltativo, rappresenta il timestamp da convertire in stringa. Se omesso viene utilizzato il time() corrente **format** è una stringa che rappresenta il formato in cui deve essere restituita la data. I formattatori utilizzabili sono in realtà moltissimi. Esempi:
 'd/m/Y' Y=anno a 4 cifre: y=anno a 2 cifre 'd-m-y'
 'H:i:s' H=0-24; h=0-12
 'Y-m-d H:i:s'

strtotime(str) inversa rispetto alla precedente. Riceve come parametro una data memorizzata sotto forma di **stringa** in the English date format (Y/m/d) e la restituisce sotto forma di **timestamp unix**.

date_create_from_format('d/m/Y:H:i:s', str);
 Simile alla precedente, consente però di specificare il formato in cui è memorizzata all'interno di **str** la data da convertire.
 Restituisce però la data sotto forma di oggetto datetime. Per ottenere il timestamp unix occorre applicare il metodo \$date->getTimestamp()

```
$date = date("Y-m-d", strtotime($myStringDate)); // anno da 2 a 4 cifre
```

La differenza fra due date

```
$date1 = new DateTime("2007-03-24 10:15:20");
$date2 = new DateTime("2009-06-26 11:16:10");
// L'ordine con cui si utilizzano le date è irrilevante !
$interval = $date1->diff($date2);
$years=$interval->y;
$months= $interval->m;
$days=$interval->d;
$hours=$interval->h;
$minutes=$interval->i;
$seconds=$interval->s;

// oppure
echo $interval->format('%h ore, %i minuti');
```

\$interval->days restituisce il numero complessivo di giorni che intercorrono fra le due date, senza suddividerlo in anni, mesi e giorni
days è però l'unica property di questo tipo. Non esistono ad es hours o minutes

Sintassi equivalente in forma procedurale:

```
$date1 = date_create("2007-03-24 10:15:20") //oppure new DateTime(".....")
$date2 = date_create("2009-06-26 11:16:10") //oppure new DateTime(".....")
$interval = date_diff($date1, $date2);
```

Per trovare la differenza in secondi fra due date sono sufficienti le seguenti istruzioni:

```
$time1 = strtotime('data1'); $time2 = strtotime('data2');
$differenceInSeconds = $time2 - $time1;
```

Variabili superglobali

Le variabili superglobali sono variabili predefinite, direttamente accessibili in qualsiasi script sempre in forma di **vettori associativi** chiave – valore. Le variabili superglobali sono

\$_SERVER che contiene tutte variabili di sistema del server

```
foreach ($_SERVER as $key => $valore)
    echo ("$key : $valore <br>");
```

le variabili **\$_SERVER** utilizzate più frequentemente sono:

REQUEST_METHOD metodo utilizzato per inviare la richiesta http (get o post)

PHP_SELF contiene la risorsa richiesta dal client

HTTP_USER_AGENT nome del browser utilizzato dal client che ha effettuato la richiesta.

REQUEST_URI contiene la risorsa completa richiesta dal client comprensiva dei parametri urlencoded

Esempio: `$url = "http://$_SERVER[HTTP_HOST]/$_SERVER[PHP_SELF]";`

\$_POST, \$_GET, \$_REQUEST, \$_COOKIE, \$_SESSION

\$_ENV contiene molte variabili d'ambiente,

Il caricamento di questo vettore deve però essere esplicitamente abilitato all'intero del file php.ini

\$GLOBALS (senza underscore). Elenco di tutte le variabili definite all'interno dell'applicazione (sia quelle utente sia alcune di quelle superglobali di sistema).

La funzione **phpinfo()** consente di visualizzare un lunghissimo elenco di variabili relative alla configurazione di PHP, del server Apache, e degli altri server.

Incorporazione di file: include e require

E' possibile includere alla pagina web un file esterno di funzioni php utilizzando le funzioni **include** o **require**. Le due funzioni sono praticamente identiche. L'unica differenza è che, in caso di file mancante:

- **require()** will produce a fatal error and stop the script. ES **require ("mioFile.php") ;**
- **include()** will only produce a warning and the script will continue

Il require in caso di file non trovato interrompe lo script generando un errore che eventualmente può essere intercettato tramite un **try - catch**

require once() fa in modo che il file venga incluso una sola volta

E' possibile includere file con qualunque estensione (typ php, html, txt).

Il simbolo @

Anteponendo il simbolo @ davanti ad una qualunque espressione/ funzione PHP, gli eventuali messaggi di errore prodotti da quell'espressione vengono soppressi:

`$conID = @mysql_connect()`

Connessione tra PHP e MySQL

L'oggetto mysqli

Le cosiddette **mysqli functions** sono semplici funzioni che consentono l'interfaccia tra PHP e MySQL. PHP è un linguaggio **procedurale** nato molto prima rispetto alla OOP. Nel momento in cui sono risultati evidenti i vantaggi della OOP (soprattutto in sede di sviluppo collaborativo), invece di riscrivere da zero l'intera architettura, come è stato fatto ad es da Microsoft con dot NET, nel caso di php sono state aggiunte semplicemente delle patch in modo da far sembrare OOP ciò che in realtà continua ad essere procedurale.

In quest'ottica le **mysqli functions** sono state sostituite dalla libreria **MySQLi** che significa **MySQL improved** e che presenta una doppia interfaccia, sia **procedurale** sia **object oriented**.

- l'approccio procedurale utilizza una sintassi molto simile alle mysqli functions originali.
- l'approccio object oriented utilizza un costruttore che restituisce un oggetto `$connection` dotato di proprietà e metodi. Per accedere a proprietà e metodi dell'oggetto si utilizza l'operatore `->` (come in C++) per il semplice motivo che si è voluto mantenere il puntino come operatore di concatenamento.

A livello di prestazioni non ci sono differenze fra la programmazione procedurale ed object oriented. La programmazione *object oriented* è però più facilmente comprensibile e riutilizzabile.

In seguito è stato aggiunto anche un altro oggetto **PDO** (Php Data Object), completamente OOP ed in grado di interfacciare non solo MySQL ma ben 12 dbms (compreso SQL Server).

Proprietà e Metodi dell'oggetto mysqli (versione Object Oriented)

`$con = new mysqli();` stabilisce la connessione con MySQL. Restituisce false se fallisce (ad es se il dbms non risponde, oppure il database indicato non è presente, o se l'utente non ha i diritti di accesso)
`$con->connect_errno` Codice di errore relativo alla mancata connessione
`$con->connect_error` Messaggio di errore relativo alla mancata connessione
`$con = mysqli_pconnect( );` stabilisce una connessione persistente che non viene chiusa né alla chiusura dello script né dal metodo `close( )`. Viene abbattuta dal MySQL server e seguito di un tempo di inattività dato dalla variabile di mysqli `wait_timeout`
`$con->host_info( )` Informazioni relativa alla connessione corrente

`$rs = $con->query($sql);` Esegue il comando SQL sulla connessione indicata e restituisce il recordset corrispondente. (che potrebbe eventualmente avere un `num_rows=0`). In caso di errore restituisce **false**. I comandi DDL e DML restituiscono **true** se ok oppure **false** in caso di errore (ad esempio per id replicato). Se non trova il record da cancellare / modificare restituisce comunque true.
`$con->errno` Codice di errore relativo all'ultimo comando SQL eseguito
`$con->error` Messaggio di errore relativo all'ultimo comando SQL eseguito
`$qta = $rs->num_rows();` restituisce il numero di record appartenenti al recordset.

`$rs = $con->multi_query($sql);` Consente di eseguire query multiple memorizzate all'interno di `$sql` e separate da punto e virgola

`$record= $rs->fetch_row( );` restituisce i campi del record corrente in un vettore **enumerativo**
`$record $rs->fetch_assoc( );` restituisce i campi del record corrente in un vettore **associativo**
`$record = $rs->fetch_array( );` restituisce i campi del record corrente in un vettore accessibile sia come vettore enumerativo sia come vettore associativo `record['nomeCampo']`. In realtà per ogni campo restituisce due property, una con chiave "0" e l'altra con chiave "nomeCampo", per cui se il risultato deve essere restituito tramite uno stream json crea soltanto problemi. Molto meglio `fetch_assoc()`.

`$data = $rs->fetch_all(MYSQLI_ASSOC);`

- restituisce l'intero recordset sotto forma di vettore enumerativo
- La costante **MYSQLI_ASSOC** fa sì che i **singoli campi** vengano restituiti sotto forma di vettore associativo, altrimenti verrebbero restituiti anch'essi sotto forma di vettore enumerativo in sequenza
- Va in errore nel caso dei comandi DML che restituiscono true/false.
- **Restituisce tutti i valori sotto forma di stringa**, anche nel caso di campi numerici.
Dovrà essere il programmatore a convertirli eventualmente in intero / float / bool

`$value = $rs->result($i, "Autore");` Restituisce il campo indicato all'interno dell'i-esimo record

`$num=$rs->affected_rows();` restituisce il numero di record modificati da un comando DML

`$id=$rs->insert_id();` restituisce il codice assegnato ad un contatore dopo un'istruzione INSERT

Esempio complete Object Oriented

```
define('DBNAME', 'db_dischi');
define('DBHOST', 'localhost');
define('DBUSER', 'root');
define('DBPASS', '');
$con = new mysqli(DBHOST, DBUSER, DBPASS, DBNAME);
if ($con->connect_errno)
    die("Errore Connessione: " . $con->connect_errno . $con->connect_error);
else
    $con->set_charset("utf8");

$sql = "select * from libri";
$rs = $con->query($sql);
if(!$rs)
    die("Errore query: " . $con->errno . "-" . $con->error);

$qta = $rs->num_rows; // property
while ($riga = $rs->fetch_assoc()) {
    echo("<TR>");
    echo("<TD>".$riga["Autore"]."</TD> <TD>".$riga["Titolo"]."</TD>");
    echo("</TR>");
}
$con->close();
```

Esempio completo procedurale

```
$con=mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);
if (mysqli_connect_errno($con))
    die ("Errore Connessione:". mysqli_connect_errno($con) . mysqli_connect_error($con));
else
    set_charset($con, "utf8");
$sql = "select * from libri";
$rs = mysqli_query($con, $sql);
if(!$rs)
    die("Errore query: " . mysqli_errno($con) . "-" . mysqli_error($con));

$qta = mysqli_num_rows($rs);
while($riga = mysqli_fetch_assoc($rs)) {
    echo("<TR> <TD>" . $riga["Autore"] . "</TD> </TR>");
}
mysqli_close($con);
```

Try and Catch

Il try and catch è un costrutto nato con la programmazione OOP. L'oggetto **mysqli**, pur presentando una interfaccia OOP, rimane di base procedurale per cui, ad esempio, nel caso di errore nella creazione di una nuova connessione, il metodo **mysqli_connect()** non genera una eccezione, ma si limita a settare le proprietà **connect_errno** e **connect_error**. Allo stesso modo il metodo **mysqli_query()**, in caso di errore, non genera una eccezione ma restituisce **null**.

Per fare in modo che i vari metodi **mysqli** generino una eccezione in caso di errore, occorre eseguire preventivamente la seguente la seguente dichiarazione:

```
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
try{
    $con = new mysqli(DBHOST, DBUSER, DBPASS, DBNAME);
}
catch (Exception $ex) {
    die ("Errore connessione db: <br>" . $ex->getMessage());
}
```

Serializzazione del recordset restituito da mySqli

Il metodo **\$rs->fetch_all()** restituisce un vettore enumerativo di record che può essere serializzato ed inviato al client

```
$data = $rs->fetch_all(MYSQLI_ASSOC);
echo(json_encode($data));
```

che equivale alle seguenti istruzioni:

```
$vect = array(); // vettore enumerativo di record
while($riga = $rs->fetch_assoc())
    array_push($vect, $riga);
echo(json_encode($vect));
```

Se si ha la certezza che il recordset contenga un solo record, si può inviare il singolo record invece che un vettore di record lungo 1:

```
$data = $rs->fetch_all(MYSQLI_ASSOC);
echo(json_encode($data[0]));
```

oppure si può utilizzare metodo **\$rs->fetch_assoc()** che restituisce una singola riga del recordset (quella attualmente puntata dal cursore)

```
if ($rs->num_rows == 1){
    $data = $rs->fetch_assoc();
    echo(json_encode($data));
}
```

Volendo invece restituire un singolo campo:

```
$data = $rs->fetch_all(MYSQLI_ASSOC);
echo(json_encode($data[0]['cod']));
```

Metodi di istanza e Metodi di classe

self:: consente di accedere ai membri statici di una classe

\$this-> consente di accedere ai membri di istanza.

notare che dopo **\$this->** il nome della variabile va scritta senza dollaro

Applicazioni SSR (multi page) e Applicazioni CSR (single page)

Applicazioni SSR (multi page)

Come detto all'inizio, originariamente le applicazioni web erano di tipo **SSR** = **Server Side Rendering** cioè il server, in corrispondenza di ogni richiesta, ricostruiva dinamicamente l'intera pagina html prima di inviarla al client, aggiornando ogni volta i dati necessari. Approccio abbastanza pesante per due motivi:

1. Quando i contenuti sono diventati principalmente multimediali (immagini, video) ricostruire ogni volta l'intera ha iniziato a diventare troppo oneroso.
2. In caso di richiamo della stessa pagina, la gestione dei dati diventa piuttosto complicata perché il server deve ricostruire la pagina da zero, cercando però al contempo di mantenere le informazioni eventualmente già inserite dall'utente, informazioni che altrimenti andrebbero perse.

In quei tempi le applicazioni erano principalmente di tipo **multi-page** in cui una pagina iniziale, in corrispondenza di un submit, richiama una'altra pagina passandogli le informazioni necessarie

Applicazioni CSR (SPA single page application)

Alla luce di questi problemi (specialmente il primo), negli anni 2000 nasce la tecnologia **Ajax** in cui l'aggiornamento dei dati viene demandato al client (**CSR** = **Client Side Rendering**)

Il client scarica la pagina dal web server una sola volta. Dopo di che, tramite Ajax, richiede i dati solo quando gli servono ed aggiorna dinamicamente la pagina, senza doverla ricaricare completamente.

Con l'avvento di **Ajax** la tendenza è diventata sempre più quella di sviluppare applicazioni **Single Page** in modo da scaricare l'intera pagina HTML una sola volta all'inizio e poi visualizzare di volta in volta le varie sezioni **senza dover passare dati da una pagina all'altra** e quindi con una navigazione molto fluida. Sono applicazioni **single-page** **google**, **gmail**, **facebook**.

Lo svantaggio principale del single page è legato **all'indicizzazione da parte dei motori di ricerca**.

- In una applicazione multi page quando si ricerca una certa informazione, i motori di ricerca rimandano direttamente alla pagina contenente quella informazione.
- In una applicazione single page questo è molto più complicato perché il motore di ricerca dovrebbe individuare quale sezione visualizzare all'interno di una stessa pagina.

Applicazioni SSR : Concetto di Web Form e pulsante di submit

Una **Web Form** è costituita da un tag `<form>` con all'interno un insieme di controlli per l'inserimento di informazioni. Serve tipicamente per trasmettere i dati da un client ad un http server.

- Il pulsante `<input type="submit">` **avvia la trasmissione al server del contenuto di TUTTI i controlli presenti all'interno della form.**
- L'attributo **ACTION** del tag FORM indica la risorsa da richiedere al server in corrispondenza del SUBMIT (tipicamente una pagina HTML o PHP).
In assenza dell'attributo ACTION il browser richiama automaticamente la pagina stessa.
- L'attributo **METHOD** del tag FORM indica la modalità di passaggio dei parametri e può assumere i valori **GET** (default) oppure **POST**. I metodi PATCH/PUT/DELETE sono utilizzabili soltanto in javascript relativamente alle richieste ajax

La trasmissione dei parametri nelle web form

- Nel caso di un submit di tipo **GET**, il browser **accoda automaticamente alla URL il contenuto di tutti i controlli presenti all'interno del tag form**, in cui:
 - Come **chiave** viene utilizzato il **name** del controllo
 - Come **valore** viene utilizzato il **value** del controllo

`www.miosito.it/pagina2.html?txtNome=mario&txtCognome=rossi&txtEta=16`

In questo caso i parametri risulteranno visibili sulla barra di navigazione accodati all'indirizzo. Questa tecnica è molto comoda per richiamare direttamente dalla barra di navigazione una pagina web che prevede dei parametri e consente di copia / incollare una URL completa comprensiva dei parametri. Le richieste effettuate tramite il tag `<a>` sono sempre richieste di tipo HTTP GET.
- Nel caso di un submit di tipo **POST** il contenuto di tutti i controlli presenti all'interno della form viene inviato al server inserendoli **all'interno del body della HTTP REQUEST**, in modo che non siano visibili sulla barra di navigazione. Maggiore riservatezza. Il formato utilizzato in questi casi è il JSON.
- Tutti i controlli devono essere inseriti all'interno di un tag **form** che è **obbligatorio**. **In assenza del tag form il submit NON viene eseguito !!**
- Se un controllo è **privo del name** o **disabilitato** il contenuto NON viene inviato al server.
- Nel caso del **ListBox**, viene inviato al server il value del ListBox che contiene il value della voce selezionata. Se le varie voci non hanno i values, viene automaticamente inviato al server il contenuto html della voce selezionata.
- Nel caso delle chiamate **'post'** è possibile **concatenare manualmente dei parametri GET in coda alla risorsa indicata all'interno della action**. In questo modo i controlli della form verranno passati in modalità POST, mentre quelli concatenati alla URL verranno passati in modalità GET.
- In caso invece di chiamata **'get'**, nel caso di axios e \$.ajax i **parametri della form sovrascrivono i parametri concatenati manualmente, per cui il passaggio 'misto' non è possibile**
- Gli attributi **action** e **method** della form possono essere eventualmente scritti come attributi del **pulsante di submit** nel modo seguente:

```
<button type='submit' formmethod='get' formaction='pag2.html'> </button>
```

Esecuzione del Submit tramite procedura javascript

Volendo validare i valori inseriti prima di eseguire il submit, è possibile sostituire il pulsante di submit con un **normale button** che richiami una procedura java script la quale potrà eseguire tutti i controlli del caso e, in caso di esito soddisfacente, eseguire il submit al server nel modo seguente:

```
let form = document.getElementById("form1");
form.method="post";
form.action="elabora.php?operazione=add";
form.submit();
```

Esecuzione del Submit tramite procedura jQuery

```
let _form=$(this).parent("form"); // genitore diretto
_form.prop("method", "post");
_form.prop("action", "pagina2.php");
_form.submit();
```

Lettura lato server dei Parametri Get e Post

PHP restituisce :

- i parametri GET all'interno di un vettore associativo **\$_GET**.
- i parametri POST all'interno di un vettore associativo **\$_POST**. Attenzione che per default PHP si aspetta i parametri POST scritti in formato URL_ENCODED e non in formato JSON
- Il vettore associativo **\$_REQUEST** contiene l'elenco dei parametri sia GET che POST.

Per conoscere il metodo utilizzato dal client si può utilizzare la variabile globale

\$_SERVER[REQUEST_METHOD] che restituisce un valore automaticamente convertito in maiuscolo:
`if ($_SERVER[REQUEST_METHOD] == "GET" )`

Lato server in caso di parametri, la **PRIMA cosa da fare è sempre** verificare se i parametri sono settati

```
if (isset($_GET['txtNome'])) $nome = strtolower($_GET['nome']);
```

Se i parametri non sono settati si segnala l'errore e si termina (altrimenti lo script andrebbe in errore)

Redirect

Il redirect lato server è una operazione che consente al server di restituire una pagina differente rispetto a quella richiesta. Molto utilizzato nei sistemi SSR per il login (se il login va a buon fine -> redirect alla prima pagina reale del sito). In php per eseguire il redirect si usa la funzione **header()** che, tramite l'utilizzo dell'oggetto **location**, richiede al browser il caricamento di una pagina diversa rispetto a quella attuale:

Come path si può passare o un **percorso assoluto** (che deve iniziare con uno **/**), oppure un **percorso relativo** a partire dalla posizione della pagina corrente.

```
header("location: pagina2.php"); // percorso relativo
header("location: C:/XAMPP/HTDOCS/4B/ese03/pagina2.php"); // assoluto
header("location:" . __DIR__ . "/pagina2.php")
```

Notare che:

- La pagina viene comunque inviata al client che leggerà la header ricevuta e richiederà la nuova pagina al server
- le echo() antecedenti al **redirect** non vengono visualizzate dal browser.
- **Il redirect non termina l'elaborazione dello script in corso** per cui se più avanti nel codice c'è un'altra istruzione di redirect, la successiva copre la precedente, che dunque NON verrà eseguita

Note sui Parametri Vettoriali e Plain Object nelle chiamate GET

Parametri vettoriali

Se più controlli presentano lo stesso name (es **checkbox**) il browser accoda i valori alla queryString come parametri indipendenti: Esempio: **hobbies=calcio&hobbies=cinema&hobbies=musica**. Quando un server va a leggere un parametro definito più volte, generalmente legge **soltanto l'ultimo valore** che in pratica sovrascrive i precedenti (comportamento anche dei server php)

Nel caso di php, affinché il server possa leggere tutti i valori, occorre aggiungere nell'html, al termine del name, una aperta e chiusa quadra es **chkHobbies[]**. I parametri url-encoded il cui nome termina con **[]** vengono aggregati dal server php all'interno di un unico vettore enumerativo contenente tutti i vari valori (anche nel caso in cui il valore sia uno solo).

Lato PHP, quando si accede ad un vettore enumerativo, le parentesi quadre dopo il nome della variabile devono essere omesse:

```
$hobbies = $_GET["chkHobbies"]
```

\$hobbies sarà un normale vettore enumerativo

Nota: Quando si utilizza axios per inviare una richiesta ajax di tipo GET, se ad un parametro si assegna un valore vettoriale, come ad esempio

```
axios.get('/api', {params: {chkHobbies:['sport','cinema','musica']} });
```

axios automaticamente aggiunge le parentesi quadre in coda al nome del parametro che viene tradotto in URL-encoded nel seguente modo:

```
?chkHobbies[]=sport&chkHobbies[]=cinema&chkHobbies[]=musica
```

In alternativa si potrebbero usare nomi diversi oppure comporre manualmente una stringa con i diversi valori separati da virgola: {vet: "1,2,3"}. Il server potrà poi fare una explode()

Plain Object (JSON con un solo livello di annidamento)

Le webform HTML non supportano strutture annidate, che però possono essere usate nelle chiamate ajax :

```
axios.get('/api', {params: { "elem":{"a":1, "b":2 } } });
```

axios automaticamente trasforma questo plain object nel modo seguente URL encoding

```
?elem[a]=1&elem[b]=2
```

Che rappresenta esattamente la sintassi attesa da PHP il quale restituirà dentro **elem** il corrispondente vettore associativo :

```
$elem = $_GET["elem"]
```

Questa notazione non corrisponde ad esempio alla sintassi attesa da json-server che utilizza la **dot notation** e quindi si aspetta

```
{ "elem.a":1, "elem.b":2 } che in URL encoding diventa
```

```
?elem.a=1&elem.b=2
```

Lettura dei parametri POST direttamente nello stream d input

Nel caso di chiamate ajax di tipo PUT o PATCH, php non restituisce da nessuna parte i relativi parametri (né dentro \$_POST né dentro \$_REQUEST), per cui occorre andare a leggerli manualmente nel body della http request:

```
$str = (file_get_contents('php://input')); // stringa
$params = json_decode($str, true);
if(is_array($params))
    $_REQUEST = array_merge($_REQUEST, $params);
```

Applicazioni CSR : Creazione di un SERVIZIO

Apache è nato per inviare pagine html/php. Volendo creare un rest service che restituisca un semplice JSON, è sufficiente utilizzare adeguatamente la funzione **echo** in modo che vada a scrivere dentro la pagina NON tag html ma uno stream json che sarà restituito al termine del servizio.

Siccome però il rendering finale viene inviato sempre come “pagina”, la pagina dovrà **contenere un UNICO SCRIPT PHP** facendo molta attenzione a non lasciare spazi **prima** e **dopo** dello script, spazi interpretati come caratteri html che verrebbero aggiunti alla stringa di risposta.

Intestazione di un servizio

la prima riga di un servizio deve sempre essere definire una intestazione che specifichi

- il tipo di risposta (**Content-type**) che verrà ritornata al client
- il formato di memorizzazione dei caratteri (**charset**) della risposta stessa.

```
header('Content-type: application/json; charset=utf-8');
```

Questa riga **NON serve** nel caso delle pagine in quanto content-type e charset sono già definiti nel filehtml

Gestione degli errori lato server

Gli errori lato server possono essere gestite in tre modi:

1) errore grave con terminazione dello script

```
http_response_code(503);
die(json_encode(['ok' => false, 'errore' => "Errore connessione db: " .
    $ex->getMessage() ]));
```

2) Errore grave senza terminazione dello script

```
http_response_code(401);
return json_encode(['ok' => false, 'errore' => 'Credenziali non valide']);
```

3) Errore non grave (semplice messaggio)

```
return json_encode(['ok' => false, 'errore' => 'Disco non disponibile.']);
```

Grazie al fatto che all'inizio di ogni servizio php aggiungiamo sempre la precedente intestazione

Content-type: application/json, il browser quando riceve la risposta parsifica **SEMPRE** il json ricevuto, anche in caso di errore (a differenza di express che invece, per default, imposta application/json SOLO in caso di response-code == 200).

Principali status code ritornati dalle http response

Se si richiede un dominio inesistente, il browser risponde con **"Impossibile raggiungere il sito"**

Viceversa se il dominio esiste e non si verificano errori, il web server restituisce automaticamente uno status_code = **200** che significa OK. In caso di errore occorre impostare esplicitamente un codice di err:

```
http_response_code(500);
```

Error 404 Page Not Found Significa dominio esistente, ma risorsa non trovata

Error 401 Unauthorized Significa credenziali non valide

Error 403 Forbidden (credenziali ok ma accesso non consentito (es sessione o token scaduti)

Error 500 Internal Server Error Significa errore nel codice lato server

Error 503 Database Connection Error

Error 400 Bad Request Il server non riesce ad interpretare la richiesta pr un errore nella sintassi dei parametri. Ad esempio per un **errore sintattico**, oppure un **parametro mancante**, oppure di un parametri di **tipo diverso** rispetto al tipo atteso (numerico / stringa)

Error 422 Unprocessable Entity Errore logico sui parametri. Ad esempio un parametro fuori dal range consentito, (eta < 18). The server understands the content type of the request entity and the syntax of the request entity is correct (thus a 400 (Bad Request) status code is inappropriate) but was unable to process the contained instructions.

path

relativi : `require_once("../MySQL.php");`

assoluti: `require_once( __DIR__ . "../MySQL.php");`

come separatore si usa comunque **SEMPRE** lo slash

Gestione Caratteri speciali e apici singoli/doppi

Lettura da database

Stabilita la connessione con il DB occorre indicare a PHP il set di caratteri (**charset**) usato da MySql (quello che compare in fase di creazione del DB) in modo da poter leggere correttamente tutti i caratteri speciali.

```
$con->set_charset("utf8mb4");           // utf8 su 4 bytes
$con->query("set names 'utf8mb4'");      // equivalente
```

Visualizzazione dei dati nella pagina HTML

Quando un valore letto da database viene utilizzato come **valore di un attributo html** (ad esempio come **value** di un textbox) eventuali apici o doppi apici interni alla stringa possono interferire con l'apice del value medesimo e vengono interpretati come identificatore di chiusura della stringa interna.

Es: `<input type='text' value='Gigi D'Alessio' >`

Questa situazione può essere risolta usando le funzioni `htmlentities` (che converte tutti gli html-entities) oppure `htmlspecialchars` che converte in html-entities SOLO i caratteri speciali come `>` in `>`; **apice** in `'`; L'opzione `ENT_QUOTES` converte sia apici singoli che doppi.

```
$value= htmlentities($value, ENT_QUOTES);
$value= htmlspecialchars ($value, ENT_QUOTES);
echo ("<input type='text' value='$value'>"); // Il risultato sarà il seguente
<input type='text' value='Gigi D&#039;Alessio' >
```

Scrittura sul database: il problema degli apici

Eventuali apici singoli all'interno di una stringa possono creare problemi in fase di inserimento nel DB

```
$s = $con->real_escape_string($s);
```

antepone un backslash davanti a: apice singolo, apice doppio e backslash.

Questa funzione dipende dall'oggetto connection ed accetta come parametro SOLO stringhe.

Scrittura sul database: il problema del SQL injection

SQL injection è una tecnica di code injection, usata per attaccare applicazioni di gestione dati, con la quale vengono inseriti delle stringhe di codice SQL malevole all'interno di campi di input in modo da modificare il significato della stringa sql. Si supponga che all'interno di un TextBox un utente inserisca `user = nn`

L'applicazione potrebbe impostare una query del tipo:

```
select * from utenti where user = '$user'; // che diventa
select * from utenti where user = 'pippo';
```

Se l'utente invece di pippo scrive nel textbox `xx' or 'A' = 'A' --`

dove -- significa che tutto ciò che segue è commentato, il comando sql precedente diventerebbe:

```
select * from utenti where user = 'xx' or 'A' = 'A' -- ';
```

Per cui la query andrebbe sempre a buon fine.

Non solo, ma se l'utente inserisse come username `xx'; DROP TABLE utenti; --`

Il comando sql precedente diventerebbe

```
select * from utenti where user = 'xx'; DROP TABLE utenti; --';
```

Per evitare sia il problema degli apici, sia l'SQL injection si può :

- **parametrizzare la query**, passando i parametri in coda
- oppure **invocare** la funzione `$con->real_escape_string($s)`;

L'oggetto SESSION

Una sessione di navigazione è una sequenza temporale di richieste http correlate tra uno stesso client ed uno stesso server.

A seguito di un login, il server può creare un oggetto **Session** persistente di identificazione di quel client, in modo che, in corrispondenza delle richieste successive, il server possa riconoscere il client senza necessità di ulteriori autenticazioni. All'interno dell'oggetto **Session** il server può definire delle **variabili di sessione** relative a quel client.

Creazione di una SESSIONE : il metodo session_start()

- Quando il server riceve una nuova richiesta di login da un client (sconosciuto e anonimo), il metodo **session_start()**, consente di creare sul server un nuovo **oggetto Session** relativo a quel client, caratterizzato da un **sessionID** univoco, e a salvarlo sul server stesso (di solito all'interno della cartella /tmp o in un percorso specifico definito all'interno di php.ini).
- Inoltre, sempre il server, **crea automaticamente anche un cookie**, avente nome fisso **PHPSESSID** in cui salva il **sessionID** relativo all'oggetto SESSION appena creato. Questo cookie viene poi inviato al client insieme ai dati o alla pagina richiesta, ed il browser del client provvederà automaticamente a rimandarlo indietro in corrispondenza di ogni richiesta successiva verso quel server.
- Sull'oggetto SESSION php assegna una durata di default pari a 1440 sec (24 minuti a partire dall'istante di creazione). Se per 24 minuti non arrivano richieste dal client, l'oggetto SESSION viene automaticamente rimosso.
*Questo valore può essere modificato all'interno del file php.ini : **session.gc_maxlifetime = 1440** oppure da codice prima di richiamare session_start() : **ini_set('session.gc_maxlifetime', 3600);***
- Il cookie invece viene creato con un lifetime = 0 il che significa che il browser manterrà vivo il cookie fino alla sua chiusura. Anche sul cookie si può personalizzare il lifetime da codice (sempre prima di richiamare session_start()) mediante l'istruzione **session_set_cookie_params(3600);** Impostando un tempo diverso da 0, questo fa sì che, alla chiusura del browser, i cookies vengano salvati sul disco del client con relativo mantenimento della sessione anche in corrispondenza della chiusura e riapertura del browser. Allo scadere del timeout, il browser non invierà più il cookie.
- In corrispondenza di ogni richiesta successiva il server, sempre tramite il metodo **session_start()**, automaticamente va a leggere il cookie denominato **PHPSESSID** contenente il **sessionID** mediante il quale accede all'oggetto Session contenente tutte le informazioni di sessione.
- Se la sessione è scaduta, il metodo session_start() ne crea automaticamente una nuova, **per cui è compito del programmatore salvare all'interno della Sessione delle informazioni che consentano di capire che la sessione è scaduta costringendo l'utente ad eseguire nuovamente in login**

Nota 1 : L'istruzione `ini_set('session.gc_maxlifetime', 600);`

fa sì che, allo scadere del tempo indicato (10 minuti), php provvederà a contrassegnare la sessione come "garbage". Di conseguenza la sessione non viene subito rimossa dalla memoria per cui, utilizzando soltanto questa impostazione, la durata effettiva della sessione diventa sostanzialmente non deterministica, perché la sessione viene mantenuta finché non si attiva il garbage collector.

Nota 2 : Sul cookie è buona regola assegnare lo stesso tempo di vita della sessione

- Se il tempo di vita del cookie è **maggiore** rispetto alla session non è un problema. Anche se il cookie rimane 'vivo' sul browser ma la sessione è scaduta, quando il cookie relativo al session ID verrà inviato dal browser al server, il session_id non verrà più riconosciuto come valido.
- Se il tempo di vita del cookie è **minore** rispetto alla session, in corrispondenza della prossima chiamata il browser NON invierà più il cookie ed il server, in corrispondenza di una richiesta priva di cookie, non riuscirà ad agganciare la sessione che risulterà undefined.

Gestione delle variabili di sessione

Come detto, all'interno dell'oggetto Session il programmatore può definire delle **variabili Session** specifiche per il singolo utente. Le variabili di sessione, essendo memorizzate lato server, mantengono il valore tra una chiamata e l'altra fino a quando la sessione rimane attiva, cioè fino a quando:

- La sessione viene esplicitamente **terminata** dal server a seguito di un logout del client
- Scatta il session-timeout

Dopo aver acceduto all'oggetto SESSION tramite il metodo **session_start()**, diventa possibile leggere/scrivere variabili di sessione con la solita sintassi dei vettori associativi:

```
$_SESSION["nome"] = "pippo";  
$nome = $_SESSION["nome"];
```

Terminazione esplicita di una Sessione : LOGOUT

Per terminare una sessione occorre eseguire, lato server, le seguenti tre operazioni:

```
session_start();    // accede all'oggetto session. Indispensabile !!  
session_unset();    // rimuove tutte le variabili session  
session_destroy();   // rimuove l'oggetto session
```

Senza l'**unset()** le variabili di sessione rimarrebbero comunque accessibili fino al termine dello script.

Quando si esegue un nuovo login, per essere sicuri di creare una nuova sessione, si può usare il metodo `session_regenerate_id(true)`; che di fatto termina una eventuale sessione ancora aperta.

Gestione manuale della durata di Sessione

Invece di usare **'session.gc_maxlifetime'**, si può creare all'interno dell'oggetto **S_SESSION** una chiave **scadenza** creata in fase di login e poi gestita ed aggiornata manualmente.

All'inizio di ogni servizio, si può aggiungere il seguente codice in cui :

- se la chiave **scadenza** è inesistente oppure scaduta, si risponde con un errore 403 in corrispondenza del quale il client verrà rimandato al login.
- se invece la sessione esiste e non è scaduta, la sua scadenza viene aggiornata, aggiornando di conseguenza anche il cookie

```
function checkSession() {  
    session_start();  
    if (isset($_SESSION['scadenza']) && time() < $_SESSION['scadenza']) {  
        $_SESSION['scadenza'] = time() + 600; // 10 min  
        setcookie(session_name(), session_id(), $_SESSION["scadenza"], "/");  
    }  
    else {  
        session_unset();  
        session_destroy();  
        setcookie(session_name(), '', -1, '/');  
        http_response_code(403);  
        die("Sessione scaduta");  
    }  
}
```

Il metodo **session_id()** disponibile solo in lettura, restituisce l'ID univoco della sessione corrente

Il metodo **session_name()** disponibile solo in lettura, restituisce il nome del cookie che verrà utilizzato per salvare il sessionID (che, nel caso di php, è sempre **PHPSESSID**).

Gestione delle variabili SESSION nelle chiamate Ajax

La validazione tramite session è stata molto utilizzata nelle applicazioni **SSR** ma va contro i principi base dei servizi REST, secondo i quali il server non deve mantenere memoria degli accessi precedenti (servizi stateless). Ma soprattutto appesantisce esageratamente ed inutilmente il server, per cui sono nate nel corso degli anni altre soluzioni migliorative come ad esempio **JWT**.

Il suo utilizzo in un sistema REST è comunque possibile procedendo nel seguente modo:

- la pagina **index.html**, in corrispondenza del prima richiesta ajax, in caso di 403 rimanda a login
- **login.html**, in corrispondenza dell'OK, richiama un servizio **login.php** che, in caso di credenziali valide, imposta la **Sessione** e restituisce un messaggio di "OK"
- Ogni servizio lato server richiama una procedura **controllaSession()** che esegue il codice precedente verificando l'esistenza e la validità della Session
 - in caso di credenziali valide, aggiorna la session e restituisce i dati richiesti
 - in caso di credenziali non valide il servizio restituirà al client un codice **403** di sessione scaduta.
- In caso di codice 403 la pagina corrente provvede a richiamare la pagina login.html

L'unico neo è che all'avvio per un attimo viene visualizzata index.html e subito dopo login.html. Questo problema può essere risolto utilizzando uno **splash screen** che rimane visualizzato finchè non arriva la risposta del server. Solo in caso di "ok" si provvederà a visualizzare il contenuto della pagina html.

Richiami sulla gestione dei cookies in PHP

La sintassi standard per la scrittura dei cookies è la seguente:

```
session_start();  
setcookie(session_name(), session_id(), time()+3600, "/");
```

La funzione **session_set_cookie_params()**

Questa funzione deve essere utilizzata *immediatamente prima* di **session_start()** e serve a modificare i parametri di default utilizzati dal server per l'invio dei cookies di sessione, **in particolare il tempo di vita del cookie**. Presenta gli stessi parametri del metodo **setcookie()**.

Notare che però ha effetto soltanto all'interno della pagina in cui si trova.

```
session_set_cookie_params(600);  
session_set_cookie_params(600, "/", "localhost", false, true);  
session_start();
```

Browser con cookies disabilitati

Anche se il browser dovesse avere i cookies disabilitati, il processo precedente può funzionare comunque, facendo uso di una tecnica detta di **URL rewriting** o **cookies munching** (sbriciolamento dei cookies)

A tal fine, all'avvio, occorre eseguire le seguenti impostazioni (tutte di default) :

```
ini_set("session.use_cookies", 0);  
ini_set("session.use_only_cookies", 0);  
ini_set("session.use_trans_sid", 1);  
ini_set("session.cache_limiter", "");
```

Con queste impostazioni tutte le URL interne alle pagine verranno **automaticamente** completate aggiungendo in coda il session ID corrente opportunamente codificato.

```
echo '<a href="pagina.php?' . SID . '">Vai alla pagina</a>';
```

Per quanto riguarda le chiamate ajax la gestione è più complicata. Il server può scrivere il session_id in coda alla pagina tramite : **echo json_encode(['session_id' => session_id()]);**
javascript può leggere questa informazione dalla pagina ed accodarla ad ogni chiamata ajax.

Lettura delle impostazioni dei cookies di sessione

Per leggere le impostazioni correnti relative ai cookies di sessione è disponibile anche il metodo:

```
$currentCookieParams = session_get_cookie_params();  
session_set_cookie_params(  
    $currentCookieParams["lifetime"],    // o altro valore  
    $currentCookieParams["path"],        // o altro valore  
    $currentCookieParams["domain"],      // o altro valore  
    $currentCookieParams["secure"],      // o altro valore  
    $currentCookieParams["httponly"]    ); // o altro valore
```

La funzione md5()

Tecnica utilizzata per evitare che estranei possano intercettare una password attraverso la rete. Invece di trasmettere la password si calcola la sua impronta MD5 (irreversibile) e si trasmette in rete e si memorizza all'interno del DB l'impronta MD5. L'impronta md5 è una sequenza di **16 bytes binari**, generalmente restituiti come stringa di 32 caratteri esadecimali-

```
string md5(string $str)
```

Esempio:

```
$str = 'apple';  
if (md5($str) == '1f3870be274f6c49b3e31a0c6728957f')    .....
```

La funzione sha256

```
$password = hash ("sha256" , $password, false); // true=return binary data
```

Invio di una mail

La funzione `mail()` nativa di PHP rappresenta la strada più semplice, ma necessita di un mail server locale. Comoda per creare una form di invio mail al server che ospita la pagina corrente (alter vista)

Molto più generale è la classe `PHPMailer` disponibile su [github](#) che consente di:

- utilizzare in modo trasparente qualsiasi server SMTP
- formattare il contenuto della mail
- aggiungere allegati che vengono automaticamente convertiti in formato base64

Per poter inviare mail tramite `PHPMailer` occorre innanzitutto abilitare il proprio **mail server** all'invio di mail ricevute da terze parti. PHPMailer supporta

- sia l'autenticazione semplice di tipo **Auth** basata sull'invio di username e password
- sia l'autenticazione più complessa e sicura **OAuth2** basata sull'invio di un token

Nel caso di Gmail per abilitare l'autenticazione semplice occorre andare su:

impostazioni / **Visualizza Tutte le impostazioni** / **Account** /
/ **Impostazioni account Google** / **Sicurezza** /
/ scorrere nella pagina e andare alla sezione **Accesso App meno sicure**

Questa funzionalità avrebbe dovuto essere disabilitata a partire dal 30/5/2022. In realtà è stata mantenuta fino al 01/05/25 per i soli account aziendali (del tipo vallauri.edu) per quali si supponeva che l'azienda fosse garante dei propri account. In data 01/05/25 in corrispondenza dell'invio della mail si verifica l'errore

CLIENT: 535-5.7.8 Username and Password not accepted.

che significa che non è più possibile utilizzare la password dell'account ma occorre creare sempre dalla **Console di Sicurezza** una nuova Password per le App in modo da usare questo token senza esporre la password del proprio account.

Per creare una password per le applicazioni occorre:

- Andare su <https://myaccount.google.com/security> che apre direttamente la finestra **Sicurezza** di Impostazioni account Google
- Andare nella sezione "Come accedi a Google" e
- Attivare **Verifica in due passaggi (2FA)** che, quando si accede al proprio account google, invierà una notifica al telefono con la domanda "Sei proprio tu ?"
Senza questa attivazione non è possibile creare un nuovo token per le app.
- Stranamente il link **"Password per le app"** non viene visualizzato all'interno della scheda sicurezza (anche se la 2FA è attiva). Occorre pertanto utilizzare il seguente link:
<https://myaccount.google.com/apppasswords>

← Password per le app

Le password per l'app ti consentono di accedere al tuo Account Google su app e servizi meno recenti che non supportano i moderni standard di sicurezza.

Le password per l'app sono meno sicure dell'utilizzo di app e servizi aggiornati che usano standard di sicurezza moderni. Prima di creare una password per l'app, verifica se l'app ne ha bisogno per effettuare l'accesso.
[Scopri di più](#)

Nessuna password per l'app disponibile.

Per creare una nuova password specifica per l'app, digita un nome di seguito...

Nome dell'app
phpMailer

Crea

Assegnare il nome **phpMailer** o poi CREA.

Il nome **phpMailer** serve solo ed esclusivamente a scopo identificativo per identificare il token e capire a che a che cosa serve quel token.

Salvare il token di 16 caratteri **eliminando gli spazi**.

Il token non verrà più visualizzato in futuro. Dovrà eventualmente essere eliminato e ricreato.

All'interno dell'applicazione, al posto della password occorre impostare questo token

Utilizzo della libreria PHPMailer

Una volta scaricato il progetto copiare la cartella src all'interno del proprio progetto

In realtà i files necessari al progetto sono soltanto due: `phpmailer.php` e `smtp.php`

```
require './src/PHPMailer.php';
require './src/SMTP.php';

$mail = new PHPMailer\PHPMailer\PHPMailer(true); // true abilita le eccezioni
$mail->IsSMTP(); // enable SMTP
$mail->SMTPDebug = 1;
$mail->Debugoutput = 'html'; // formato di visualizzazione dei msg di debug
```

La terza riga

```
// 0 = off (for production use)
// 1 = Abilita la visualizzazione di errori e messaggi
// 2 = Abilita la visualizzazione solo dei messaggi
```

Impostando 1 oppure 2 il metodo finale `send()` sovrascrive la `echo()` utente e restituisce l'elenco di tutti i messaggi scambiati tra il servizio PHP ed il server SMTP

Configurazione per l'accesso al server SMTP

```
$mail->Host = 'smtp.gmail.com';
```

Per l'invio della mail sono possibili tre protocolli :

SMTP (porta **25**) . Utilizzato tra mail server e mail server. Accetta mail senza autenticazione

SMTP over SSL (porta **465**) basato su cifratura SSL

SMTP over TLS (porta **587**) basato su cifratura TLS preferibile rispetto a SSL perchè presenta alcune opzioni e garanzie aggiuntive.

```
$mail->SMTPSecure = 'tls'; // Options: '', 'ssl' or 'tls'
$mail->Port = 587; // Options: 25, 465 or 587
// oppure
// $mail->Host = "tls://smtp.gmail.com:587";
```

Impostazione delle Credenziali di accesso all'SMTP Server

```
$mail->SMTPAuth = true; // abilitazione al controllo delle credenziali
$mail->Username = "roberto.mana@vallauri.edu";
$mail->Password = "token phpMailer";
```

Disabilitazione al controllo del certificato. (peer= certificato)

indipendentemente dal protocollo in uso. *Non più necessario*

```
$mail->SMTPOptions = array(
    'ssl' => array(
        'verify_peer' => false,           // peer = certificato
        'verify_peer_name' => false,     // inutile
        'allow_self_signed' => true      // inutile
    )
);
```

Mittente

```
$mail->setFrom("roberto.mana@vallauri.edu", 'robertoMana');
```

// Il primo parametro rappresenta l'indirizzo del mittente.

// Il secondo parametro, facoltativo, rappresenta un alias che, all'interno del client di posta del destinatario, sostituirà l'indirizzo mail del mittente.

Destinatario

```
$mail->addAddress('ing.mana@tiscali.it');
$mail->addCC('ing.mana@tiscali.it');
$mail->addBCC('ing.mana@tiscali.it');
```

Impostazione della Mail

```
$mail->Subject = 'Here is the subject';
$mail->isHTML(true); // Set email format to HTML
$mail->Body = 'This is the HTML message body <b>in bold!</b>';
$mail->AltBody = 'This is the body in plain text for non-HTML mail clients';
$mail->addAttachment('allegati/qrCode.png');
$mail->addAttachment('allegati/Desert.jpg');
```

Il primo parametro di addAttachment() rappresenta il nome del file da allegare.

Il secondo parametro rappresenta il nome da assegnare all'allegato (nome che 'maschera' il nome del file)

Invio

Alcuni errori provocano il passaggio al catch, in altri casi restituisce false.

La soluzione migliore è gestirli entrambi.

```
try{
    if ($mailer->send()){
        http_response_code(200);
        echo (json_encode('Message sent !'));
    }
    else {
        http_response_code(550); // mail error
        die ("Errore PHPMailer: " . $mailer->ErrorInfo);
    }
}
catch (Exception $e) {
    http_response_code(550);
    die ("Errore PHPMailer: " . $mailer->ErrorInfo);
}
```

Principali funzioni per la gestione dei files

__FILE__ contiene il path completo assoluto sul server del file corrente (a partire da httdocs)
__DIR__ contiene il path completo assoluto sul server della cartella corrente (a partire da httdocs)
getcwd(); uguale a **__DIR__**
\$dirname = dirname (\$filepath)
Restituisce il path completo assoluto sul server del file indicato, eliminando il nome contenuto dopo l'ultimo slash
\$filename = basename (\$filepath)
Restituisce il nome del file (con estensione) contenuto dopo l'ultimo slash, eliminando in pratica il path antecedente. Elimina anche l'eventuale slash iniziale.
file_exists (\$filepath)
Verifica se il file esiste all'interno del percorso relativo indicato (a partire dalla cartella corrente).
\$ext = pathinfo (\$file, PATHINFO_EXTENSION);
Estrae l'estensione del file ricevuto come parametro

Lettura e Scrittura di un file di testo

```
$handle = fopen (string $filename, string $mode)  
Apre il file nella modalità indicata che può essere: r = sola lettura, r+ = lettura e scrittura,  
w = sola scrittura (se il file non esiste lo crea automaticamente), w+ = lettura e scrittura cancellando  
completamente il vecchio contenuto, a = append in coda al file  
fgets ($handle, $n); // Legge una riga alla volta il contenuto del file. Il secondo parametro n indica  
quanti caratteri leggere a partire dalla posizione corrente. Se omissso viene letta l'intera riga  
fwrite ($handle, $line); // Accoda una riga al file di testo  
fclose ($handle);  
fseek ($handle, 10, SEEK_SET); // Mi posiziono al 10° carattere dalla posizione corrente  
  
$fh = fopen('filename.txt','r');  
while ($line = fgets($fh)) {  
    echo($line);  
}  
fclose($fh);  
  
$fp = fopen('lidn.txt', 'w');  
fwrite($fp, 'writing a line ...');  
fclose($fp);
```

Altre funzioni

\$string file_get_contents (\$filename); // Legge l'intero **file testuale** indicato e lo restituisce all'interno di una stringa.

array file (\$filename); // Legge il **file testuale** indicato e lo restituisce all'interno di un array, ogni riga in una cella successiva. Il **\n** viene mantenuto al termine della riga

Queste ultime due funzioni non hanno bisogno che il file sia aperto ma provvedono automaticamente ad aprirlo e chiuderlo.

\$img = getimagesize (\$file)

Legge il contenuto del file e restituisce sotto forma di **vettore associativo** tutte le varie caratteristiche dell'immagine (dimensione, mime/type, etc). Riconosce quasi tutti i principali formati relativi alle immagini. Se il file non contiene una immagine restituisce **false**.

Lettura e Salvataggio di un file binario lato server

Il server PHP (apache) intercetta i parametri ricevuti in formato FormData e:

- I campi di tipo stringa vengono riconvertiti in url-encoded e resi disponibili all'interno dei soliti vettori **\$_POST** e **\$_REQUEST**.
- I campi di tipo object sono resi disponibili all'interno di un vettore associativo **\$_FILES** il quale però non sarà restituito come vettore enumerativo di oggetti **File**, ma come **un unico oggetto in cui ogni chiave viene trasformata in un vettore enumerativo di valori**. Cioè in pratica la struttura dati restituita da PHP sarà la seguente

```
{
  name:["nome1", "nome2", "nome3", "nome4"],
  size:["size1", "size2", "size3", "size4"],
  type:["type1", "type2", "type3", "type4"],
  tmp_name:["tmp_name1", "tmp_name2", "tmp_name3", "tmp_name4"]
}
```

Nota1: Express restituisce invece un normalissimo vettore enumerativo di oggetti File.

Nota2:

- Se dopo la chiave **'txtFiles'** il client omette le parentesi quadre, il File sarà restituito da PHP come un singolo json. In tal caso le varie chiavi che compongono il File saranno restituite come semplici valori del json
- Se dopo la chiave **'txtFiles[]'** il client aggiunge le parentesi quadre, il File sarà restituito da PHP come un json di vettori enumerativi. Ogni chiave sarà costituita da un vettore enumerativo di valori, come indicato sopra.

Lettura e salvataggio di un File singolo

In assenza dell'attributo **multiple** l'utente può selezionare ed inviare un unico file. In tal caso :

- Se il **txtName** trasmesso dal client **NON** ha le parentesi quadre, le varie informazioni saranno disponibili in forma scalare, come indicato nel seguente esempio

```
$fileRicevuto = $_FILES["txtFiles"];
$filename=basename($fileRicevuto["name"]);
$size=$fileRicevuto["size"];
$mimeType=$fileRicevuto["type"];
$ext = pathinfo($filename,PATHINFO_EXTENSION);
$sourceFile=$fileRicevuto["tmp_name"];
$targetFile = "uploads/$filename";
move_uploaded_file($sourceFile, $targetFile);
```

La funzione **move_uploaded_file** consente di salvare il file ricevuto all'interno della web directory del server. Restituisce **true** in caso di successo oppure **false** in caso di errore.

In caso di file già esistente lo sovrascrive senza nessuna segnalazione.

Eventualmente si può controllare manualmente se il file già esiste, evitando di sovrascriverlo:

```
if (file_exists($target_file))
    echo ("Sorry, file already exists");
else{ ..... }
```

- Se invece il **txtName** trasmesso dal client **ha** le parentesi quadre, le varie informazioni saranno disponibili in forma di vettore enumerative con lunghezza 1. Ad esempio:

```
$fileRicevuto = $_FILES["txtFiles"]; // senza le quadre !
$filename=basename($fileRicevuto["name"][0]);
```

Files multipli

Se sul tag `<input type="file">` se si imposta l'attributo **multiple** occorre necessariamente impostare come **name** del controllo client un nome vettoriale del tipo `txtFiles[]`

```
<form action="upload.php" method="post" enctype="multipart/form-data">
    Select file to upload:
    <input type="file" name="txtFiles[]" multiple>
    <input type="submit" value="Upload">
</form>
```

Anche in questo caso apache restituirà un **unico Object** in cui ogni campo viene trasformato in **vettore enumerativo contenente i diversi values**

Lato server per scorrere i files si può utilizzare il ciclo indicato di seguito:

```
$fileRicevuti = $_FILES["txtFiles"]; // senza le quadre
for ($i = 0; $i < count($fileRicevuti["name"]); $i++){
    $filename=basename($fileRicevuti["name"][$i]);
    $size=$fileRicevuti["size"][$i];
    $mimeType=$fileRicevuti["type"][$i];
    $ext = pathinfo($filename,PATHINFO_EXTENSION);
    $target_file = "uploads/$filename";
    move_uploaded_file($fileRicevuti["tmp_name"][$i], $target_file);
}
```

Notare che l'accesso alla chiave `txtFile` deve essere sempre scritto SENZA le parentesi quadre
`$fileRicevuti = $_FILES["txtFiles"];`
obbligatorie invece nel caso del client.

Lettura e Salvataggio di un file base64

Un file base64 viene ricevuto come normale stringa POST. Oltre al file occorre inviare altre informazioni come Size e MimeType prima automaticamente memorizzate all'interno del FormData.

Per salvarlo su disco si può utilizzare la seguente semplice procedura:

```
function saveBase64($base64_string, $output_file) {
    $fp = fopen( $output_file, 'wb' );
    // $data[ 0 ] == "data:image/png;base64"
    // $data[ 1 ] == <actual base64 string>
    $data = explode( ',', $base64_string );
    // we could add validation here with ensuring count( $data ) > 1
    fwrite( $fp, base64_decode( $data[ 1 ] ));
    fclose( $fp );
    return $output_file;
}
```

Lettura di parametri passati con i metodi PUT PATCH e DELETE

Se sulla pagina client si imposta come method il valore PUT, eventuali parametri vengono inseriti all'interno del body della HTTP Request, esattamente come avviene con POST, però questa volta il server non provvede più a leggerli 'automaticamente' restituendoli all'interno di una apposita struttura dati, ma occorre andare a leggerli manualmente all'interno del body della HTTP Request.

Lettura lato PHP dei parametri urlencoded trasmessi all'interno del body della http-request

Si può utilizzare la seguente procedura basata sulla funzione **file_get_contents** che restituisce sotto forma di stringa l'intero "file testuale" ricevuto sullo stream di input.

```
function _parseParams() {  
    $params = Array();  
    $method = $_SERVER['REQUEST_METHOD'];  
    if ($method == "PUT" || $method == "DELETE") {  
        parse_str(file_get_contents('php://input'), $params);  
        $GLOBALS["_method"] = $params;  
        $_REQUEST = array_merge($params, $_REQUEST);  
        // oppure: $_REQUEST = $params + $_REQUEST;  
    }  
}
```

parse_str esegue uno split della stringa sulla base del carattere &, restituendo un vettore associativo (\$params) contenente tutti i vari parametri in formato chiave:valore.

`$GLOBALS["_method"] = $params;` crea una nuova variabile globale avente come nome il contenuto della variabile method, cioè `_PUT` oppure `_DELETE`

`$_REQUEST = array_merge($params, $_REQUEST)` copia i parametri anche dentro il vettore `_REQUEST` preposto a contenere tutti i parametri HTTP ricevuti insieme alla chiamata

Upload di un file in modalità PUT tramite Ajax

Il **Lato Client** è lo stesso identico che si usa per una richiesta POST.

Lato Server occorre andare a leggere manualmente i dati all'interno del body della HTTP Request. Trattandosi non di un file testuale ma di un file binario non è possibile utilizzare **file_get_contents**. Il file può essere letto sullo STDIN stream nel modo seguente:

```
<?php  
$fp1 = fopen("php://input", "r");  
$fp2 = fopen("newFile.ext", "w");  
/* Read the data 1 KB at a time */  
while ($data = fread($fp1, 1024))  
    fwrite($fp2, $data);  
fclose($fp1); fclose($fp2);  
?>
```

In realtà, prima della lettura, occorre ancora agire sul file eliminando le intestazioni relative alla codifica multipart, cosa fatta automaticamente da PHP, in caso di trasmissione POST, nel momento in cui intercetta il file e lo salva all'interno del vettore associativo `$_FILES`.

Creazione e serializzazione di uno stream XML

Per inviare come risposta uno stream xml, il modo più semplice è quello di creare una semplice **stringa xml** con concatenamenti successivi.

Volendo invece lavorare con gli oggetti, fino alla versione PHP 4, per il parsing di documenti XML si utilizzava la libreria **DOM** praticamente identica a quella di java script. Principali metodi :

```
$xmlDoc = new DOMDocument();  
$xmlDoc->load("file.xml");  
$xmlDoc->loadXML(xmlString); // parsing della stringa xml  
xmlString = $xmlDoc->saveXML(); // serializzazione dell'oggetto xmlDoc
```

In PHP 5 è stato aggiunto un nuovo strumento basato su un'interfaccia ad oggetti, SimpleXML, forse più semplice ma con un'interfaccia diversa rispetto al classico DOM.

Esempio basato su DOM library

```
<?php  
    $xmlDoc = new DOMDocument();  
    // creo una stringa XML da trasformare poi in object  
    $xml = "<dischi>";  
    $xml .= "<disco><autore>Battiato</autore><titolo>Centro</titolo></disco>";  
    $xml .= "<disco><autore>Zucchero</autore><titolo>Baila </titolo></disco>";  
    $xml .= "</dischi>";  
    $xmlDoc->loadXML($xml);  
    $root = $xmlDoc->documentElement;  
  
    // Esempio 1  
    foreach ($root->childNodes AS $disco)  
        foreach ($disco->childNodes AS $item)  
            echo($item->nodeName . " = " . $item->textContent . "<br>");  
  
    // Esempio 2  
    $disco = $root->firstChild;  
    echo($disco->nodeName . "<br>"); // disco  
  
    $item = $disco->firstChild; // autore  
    echo($item->nodeName . " = " . $item->textContent . "<br>");  
    $item = $item->nextSibling; // titolo  
    echo($item->nodeName . " = " . $item->textContent . "<br>");  
  
    // Esempio 3  
    $dischi = $root->childNodes;  
    $items = $dischi[1]->childNodes; // disco di zucchero  
    echo($items[0]->nodeName . " = " . $items[0]->textContent . "<br>");  
    echo($items[1]->nodeName . " = " . $items[1]->textContent . "<br>");  
  
    // Esempio 4 : creazione di un nuovo disco  
    $disco = $xmlDoc->createElement('disco');  
    $disco->setAttribute('tipo', 'nuovo');  
    $root->appendChild($disco);  
    $autore = $xmlDoc->createElement('autore', "Lucio Dalla");  
    $disco->appendChild($autore);  
    $titolo = $xmlDoc->createElement('titolo', "Piazza Grande");  
    foreach ($root->childNodes AS $disco) {  
        echo($disco->getAttribute("tipo") . "<br>");  
        foreach ($disco->childNodes AS $item)  
            echo($item->nodeName . " = " . $item->textContent . "<br>");  
    }  
}
```

Debug di una applicazione PHP con vscode

Step1: installazione di xdebug

xdebug.org/wizard

Nella pagina bianca che si presenta copia incollare l'intero risultato di `echo (phpinfo())`
Vengono quindi visualizzate le seguenti istruzioni per l'installazione di **xdebug**

1. Download [php_xdebug-3.2.1-8.1-vs16-x86_64.dll](https://xdebug.org/wizard)
2. Move the downloaded file to F:\programmi\xampp\php\ext, and rename it to `php_xdebug.dll`
3. Update F:\programmi\xampp\php\php.ini and add the line:
`zend_extension = xdebug`
4. Restart the Apache Webserver

In realtà le istruzioni da inserire all'interno di **php.ini** sono le seguenti:

```
[XDebug]
xdebug.mode=debug
xdebug.start_with_request=yes
zend_extension = "F:\programmi\xampp\php\ext\php_xdebug.dll"
```

La porta di ascolto è ora la **9003** (non più la 9000 di xdebug2)

Se si vuole cambiare la porta di ascolto si possono aggiungere in coda le seguenti due righe:

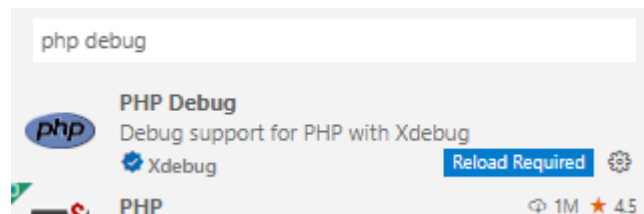
```
xdebug.client_host=host.docker.internal
xdebug.client_port=9000
```

Al termine riavviare apache

Step2: Configurazione del debugger

Nelle extensions di vscode cercare e aggiungere.

php debug



Occorre quindi creare un file di configurazione (da ripetere per ogni progetto)

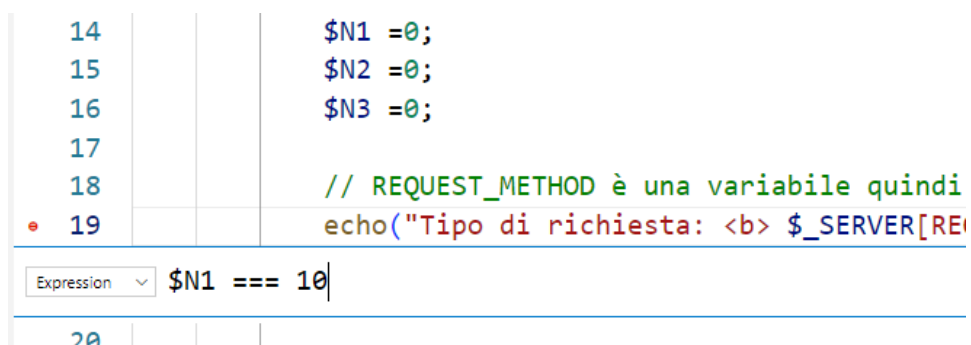
Run / Add Configuration... / PHP

Il file di configurazione viene creato correttamente e non c'è nulla da modificare.

- Mettere un breakpoint
- Aprire il browser e richiamare la pagina desiderata (oppure aprire nel browser la pagina corrente direttamente da vscode utilizzando ad esempio il plugin riportato di seguito)
- Quando il codice server arriverà al breakpoint si arresterà automaticamente.

Breakpoint condizionali

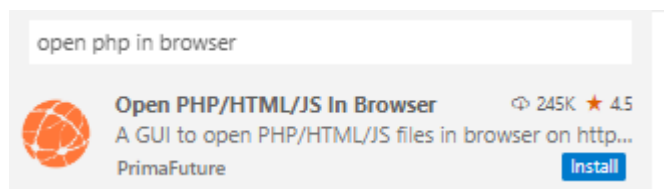
Facendo tasto destro su un breakpoint / **Edit Breakpoint**, si può inserire un cosiddetto breakpoint condizionale, cioè un breakpoint che si attiverà soltanto nel momento in cui la condizione risulterà vera, cioè, nel caso in figura, quando `$N1 == 10`



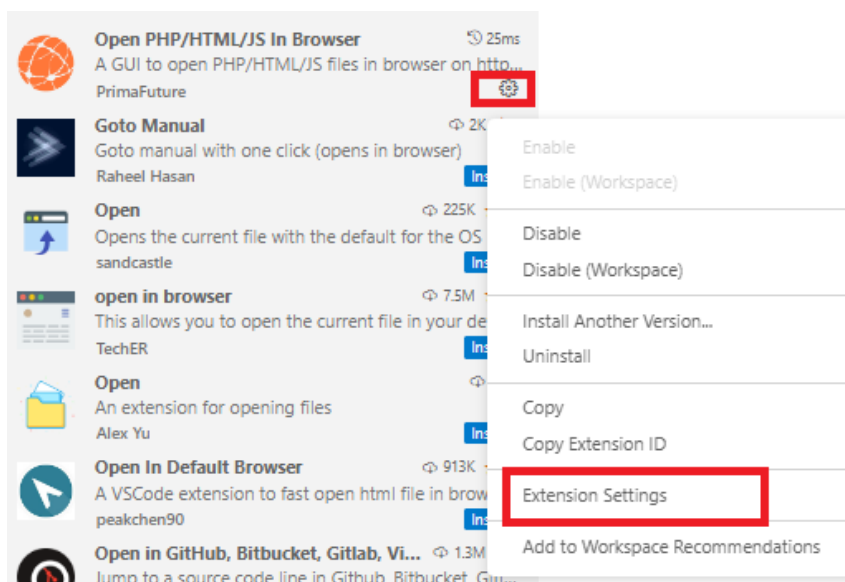
Esecuzione di una applicazione PHP direttamente da vscode

Nelle extensions di vscode cercare e aggiungere.

open php in browser



Dopo averla installata, dalla rotellina in basso selezionare *Extension Settings*



Impostare quindi la cartella di installazione di xampp, il browser da utilizzare e la URL da eseguire. Attenzione che la Document Root Folder di xampp è **case sensitive**.

Open-PHP-HTML-JS-in-browser: Custom Browser Path

Path of a custom browser executable (eg. `C:\Program Files\Browser\Browser.exe`)

Open-PHP-HTML-JS-in-browser: Custom Host

Custom localhost hostname with port number (eg. `localhost:8888`, `localhost:1234`)

Open-PHP-HTML-JS-in-browser: Custom Url To Open

Custom url to open in browser (eg. `http://${host}/${relativeDirnameDocumentRoot}/${fileBasename}}`)

Open-PHP-HTML-JS-in-browser: Document Root Folder

Base directory of your pages to serve from `http://localhost` domain (eg. `C:\xampp\htdocs\`, `/var/www/`, etc.). Autodetect by default

Open-PHP-HTML-JS-in-browser: Remember Browser Selection

☒ Remember last browser selection. Uncheck and select **Ask always...** option if you want to always choose browser to launch

Open-PHP-HTML-JS-in-browser: Selected Browser

Browser to open (**Chrome**, **Firefox**, ...)

Open-PHP-HTML-JS-in-browser: Show Status Bar Item

☒ Show the button **Open In Browser** in the vscode status bar

Open-PHP-HTML-JS-in-browser: Url To Open

Url scheme to open in browser (`http://localhost` or `file:///` or custom)

A questo punto aprire in vscode il file `index.php` e fare tasto destro / **Open PHP/HTML/JS in browser**

